TP1 — Java RMI : cabinet vétérinaire

HAI704I — Architectures logicielles distribuées · Master 1 Informatique (ICO) · Université de Montpellier · 2026–2027

Application répartie en Java RMI « pur » (sans framework) : des vétérinaires, depuis des postes clients distincts, consultent et modifient les patients et leurs dossiers de suivi hébergés par un serveur, et sont prévenus automatiquement lorsque le nombre de patients franchit certains seuils.

---

1. Le projet

1.1 Rôle de chaque projet

Trois projets indépendants, **sans `module-info.java`** :

Projet	Rôle	Contenu
`common`	Contrat partagé entre client et serveur	Interfaces distantes `IAnimal`, `IDossierSuivi`, `ICabinet`, `IObservateurCabinet` ; classes sérialisables `Espece` et `Chien`
`server`	Côté serveur	Servants `AnimalImpl`, `DossierSuiviImpl`, `CabinetImpl` ; lanceur `Server`
`client`	Côté client	Application en ligne de commande (`Main`, `ConsoleUI`, `ClientCabinet`, `ObservateurAlertes`, `ErreurCabinet`) et programmes de test

Relations de dépendance : `server` → `common` et `client` → `common`. **Le client ne dépend jamais du serveur** (il ne connaît que les interfaces et les types partagés) ; le serveur ne dépend pas du client.

1.2 Structure des sources

```
source/
├── common/src/vete/common/
│     IAnimal.java, IDossierSuivi.java, ICabinet.java, IObservateurCabinet.java,
│     Espece.java, Chien.java
├── server/src/vete/server/
│     AnimalImpl.java, DossierSuiviImpl.java, CabinetImpl.java, Server.java
└── client/src/vete/client/
    Main.java, ConsoleUI.java, ClientCabinet.java, ObservateurAlertes.java, ErreurCabinet.java
    Client.java, ClientRemplisseur.java, ExperienceA7.java    (programmes de test, voir § 2.5)
```

1.3 Dépendances et hypothèses de configuration

- Aucune dépendance externe : uniquement la bibliothèque standard Java (`java.rmi`, `java.io`, `java.util`).
- Pas d'outil de build : compilation avec `javac`, exécution avec `java`.
- Le serveur et les clients tournent sur la même machine par défaut (`localhost`). Aucune propriété `java.rmi.server.hostname` n'est positionnée.

- Pas de `SecurityManager` et pas de chargement dynamique de classes : les classes partagées doivent être présentes dans le classpath des deux côtés (c'est le rôle du projet `common`).
- Les données sont **en mémoire** : redémarrer le serveur remet l'état initial (deux patients : Moon, chien samoyède, maître Alex ; Felix, chat siamois, maître Marie).

1.3.1 Mode de registre

Registre interne : le serveur crée lui-même le registre RMI dans sa propre JVM (`LocateRegistry.createRegist` et y publie un unique objet, le cabinet, sous le nom `cabinet`. Il n'y a donc **pas** de processus `rmiregistry` séparé : le démarrage du serveur est aussi le démarrage du registre.

1.4 Organisation du client

Le client est découpé en trois responsabilités :

Couche	Classe(s)	Rôle
Interface utilisateur (CLI)	`ConsoleUI`, `Main`	Affiche le menu, lit et valide les saisies. **N'importe rien de `java.rmi`.**
Logique client	`ClientCabinet`, `ErreurCabinet`	Traduit chaque action en appels distants ; convertit les `RemoteException` en messages compréhensibles (`ErreurCabinet`) ; gère l'abonnement et la fermeture
Accès RMI	stub du cabinet, `ObservateurAlertes`	Stub obtenu auprès du registre ; `ObservateurAlertes` est un objet distant exporté par le client pour recevoir les alertes (callback)

Les alertes arrivent sur un fil du runtime RMI : la logique client les transmet à l'interface par une fonction d'affichage injectée, sans que l'interface connaisse RMI.

---

2. L'exécution

2.1 Environnement

- **JDK** : OpenJDK 21.0.12.1 (Homebrew, macOS). Le sujet demande Java 17 ou plus. Vérification : `java -version`.
- **Environnement de développement** : éditeur Visual Studio Code, avec compilation et exécution en ligne de commande (`javac`, `java`) dans le terminal. Aucun IDE de build.
- **Système** : macOS. Sous Windows, remplacer le séparateur de classpath `:` par `;`.

2.2 Compilation

Depuis la racine du dépôt (le dossier qui contient `common/`, `server/`, `client/`) :

```
javac -d common/out $(find common/src -name "*.java")
javac -cp common/out -d server/out $(find server/src -name "*.java")
javac -cp common/out -d client/out $(find client/src -name "*.java")
```

Si le dossier `source/` de l'archive est utilisé, se placer d'abord dedans (`cd source`). Les dossiers `out/` sont engendrés et ne font pas partie du rendu.

2.3 Ordre de démarrage

Un **terminal par processus**, toujours dans cet ordre :

1. **Serveur et registre** (port 1099) :

   ```
   java -cp common/out:server/out vete.server.Server
   ```

   Attendre l'affichage de `Server ready`.

2. **Premier client** :

   ```
   java -cp common/out:client/out vete.client.Main
   ```

3. **Second client** (autre terminal) :

   ```
   java -cp common/out:client/out vete.client.Main
   ```

Options du client : `Main [hote] [port]` (par défaut `localhost` et `1099`), par exemple `java -cp common/out:client/out vete.client.Main localhost 1099`.

Le serveur doit être **redémarré après toute modification du code**. Arrêt du serveur : `Ctrl+C`.

2.4 Utilisation de la CLI

Menu affiché par le client :

```
=== Cabinet veterinaire ===
1. Lister les patients
2. Rechercher un patient
3. Enregistrer un patient
4. Consulter le dossier de suivi
5. Mettre a jour le dossier de suivi
6. S'abonner aux alertes
7. Se desabonner des alertes
0. Quitter
```

Conventions de saisie : - taper le numéro de l'option puis `Entrée` ; les autres champs sont demandés un par un ; - un champ texte ne peut pas être vide ; l'espérance de vie doit être un entier strictement positif ; - toute saisie invalide (option inconnue, champ vide, nombre incorrect) affiche un message et redemande, sans arrêter le programme ; - une erreur distante (serveur arrêté, patient inconnu) affiche un message clair (par exemple `Serveur injoignable. Verifiez qu'il est demarre.`) et le menu reste utilisable.

Alertes : après l'option 6, le client reçoit à tout moment, à l'écran, un message du type `[ALERTE] seuil 100 franchi a la hausse : 100 patients` lorsque le nombre de patients franchit **100, 500 ou 1000**, à la hausse comme à la baisse.

Arrêt propre : option `0` (ou `Ctrl+D`). Le client se désabonne s'il le faut, retire son objet distant du runtime RMI et se termine de lui-même, sans `Ctrl+C`.

2.5 Programmes de test (optionnels)

- `vete.client.ClientRemplisseur` : remplit le cabinet jusqu'à 100 patients puis en retire un, pour déclencher les alertes aux deux sens.
- `vete.client.Client` : petit scénario de vérification non interactif (liste, recherche, ajout).
- `vete.client.ExperienceA7` : expérience de l'étape A7 (envoi d'une sous-classe `Chien` de `Espece`). Elle réussit car `Chien` est dans `common` ; l'échec décrit dans le compte-rendu apparaissait quand `Chien` n'existait que côté client.

Exemple pour tester les alertes : démarrer le serveur, lancer deux clients, s'abonner (option 6) dans chacun, puis lancer `java -cp common/out:client/out vete.client.ClientRemplisseur` dans un troisième terminal.

2.6 Vidéo de démonstration

Voir `video/demonstration.mp4` (ou `liens.txt` si la vidéo est hébergée à l'extérieur).